Aprendizaje profundo con la descripción de los inmuebles: alternativas y comparación

Nota de trabajo · Proyecto HE2 IA Aplicada (ECON-3527) · precios de vivienda en Bogotá

8 de noviembre de 2026 (víspera de la Semana 14)

Contenido

1. Resumen 1
2. Qué herramientas del curso tenemos a disposición 2
3. Qué contiene el texto: evidencia 2
4. Alternativas 3
5. Comparación frente al objetivo 6
6. Reglas de diseño comunes (cualquier alternativa) 7
7. Recomendación y plan hasta el 25 de noviembre 8
8. Uso de IA en esta nota 9

1. Resumen

Objetivo. El objetivo no cambia: estimar el precio publicado de una vivienda a partir de sus características. El texto de `descripcion` solo se justifica si **reduce el error fuera de muestra** frente al modelo tabular actual. Ese modelo es el Random Forest en niveles: RMSE de CV de 355 M, RMSE de prueba de 310 M y MAPE de prueba de 12.7%, frente a 433 M y 17.3% de la mejor línea base (`04_metricas_modelos.csv`). Además, no debe traer de vuelta, por la puerta de atrás, información que no corresponde a las características.

Qué encontramos en el texto (sección 3):

- Trae atributos que no están en columnas: vista, balcón, remodelación, iluminación, piso y dúplex.
- También trae riesgos: **4.4% de las descripciones dice el precio publicado** de forma literal (fuga directa del objetivo). Otras mencionan la administración, el canon de arriendo, el barrio o el estrato.
- Hay **219 viviendas de prueba con una descripción idéntica** a una de entrenamiento.
- El texto viene **cortado en 1,000 caracteres**.

Recomendación (sección 7). Tres piezas que se comparan entre sí con la misma partición:

1. **Referencia clásica de texto:** palabras clave y TF-IDF (clase de NLP, Semana 6). Mide si el texto aporta algo antes de invertir en redes.
2. **Propuesta principal de aprendizaje profundo:** vectores (*embeddings*) de un Transformer preentrenado en español, congelados, combinados con las variables tabulares en una red neuronal (MLP en PyTorch). También se prueba agregados al Random Forest. Es la propuesta de la presentación de avance, ahora con un diseño de evaluación concreto.
3. **Complemento de IA generativa:** un LLM que **convierte la descripción en características estructuradas** (vista exterior, piso, remodelado, dúplex, etc.), que luego entran al Random Forest. Es la alternativa más alineada con el objetivo paramétrico y la más interpretable.

Se descartan como línea principal tres alternativas: el ajuste fino (*fine-tuning*) de un Transformer completo, pedirle el precio directamente a un LLM y una LSTM como modelo central. Las razones están en la sección 5.

Primer paso, sea cual sea la alternativa: enmascarar en el texto el precio, la administración, el canon y los teléfonos, y reportar cuántas descripciones se modificaron.

2. Qué herramientas del curso tenemos a disposición

Mañana, lunes 9 de noviembre, empieza la Semana 14, así que el curso ya cubrió hasta la Semana 13. Esto es lo que sirve para trabajar con el texto:

Semana	Tema (programa)	Uso posible con `descripcion`
2	Regresión, regularización; árboles, RF, XGBoost	Modelos que reciben las variables de texto
3	KNN, K-means, PCA	Reducir la dimensión de TF-IDF o de los *embeddings*
4	Redes neuronales: neuronas, capas, activaciones	Red que combina texto y variables tabulares
5	Descenso de gradiente, *backpropagation*	Entrenamiento de esa red
6 (lun.)	NLP: preprocesamiento, bag-of-words, TF-IDF, similitud coseno, bigramas, LDA	Referencia clásica de texto; detectar duplicados
6 (mié.)	Representaciones vectoriales y redes para texto	*Embeddings* aprendidos o preentrenados
7	Agentes de terminal y **APIs**; redes convolucionales	LLM por API; CNN 1D sobre texto
8, 10	Redes recurrentes, LSTM, GRU	Modelo secuencial sobre las palabras
12–13	A/B testing, bandits, aprendizaje por refuerzo	Evaluación de impacto (no para el texto)

- **Transformers y LLM.** El programa los incluye en la descripción del curso (bloque II: IA generativa), pero el cronograma no los asigna a una semana. Las Semanas 15 y 16 dicen “Otras aplicaciones”.
- **Diapositivas disponibles.** En `../../Presentaciones/` llegan hasta la clase de NLP de la Semana 6. La clase 9 (bigramas con un MLP) usa **PyTorch**: es la librería de redes que ya se vio.
- **Librerías nuevas.** Según `CLAUDE.md`, toda herramienta que no se haya visto en clase se consulta con el equipo antes de usarla. Eso incluye `sentence-transformers` o `transformers` (Hugging Face) y el SDK de un proveedor de LLM. PyTorch tampoco está en `requirements.txt` todavía.

3. Qué contiene el texto: evidencia

Las cifras marcadas con † salen de una exploración preliminar no versionada (expresiones regulares sobre la base limpia y la partición actual). Son aproximadas. **Antes de llevarlas a la presentación**, hay que pasarlas a un script numerado (por ejemplo, ampliando `07_perfil_descripcion.py`), como exige la regla de que todo número sale de una corrida. Las demás cifras salen de `07_perfil_descripcion.csv`.

Cobertura y largo

- Todas las 26,900 viviendas tienen descripción. La mediana es de 75 palabras (percentil 90: 134).
- † El texto **viene cortado en 1,000 caracteres** (520 descripciones llegan al tope; el máximo es de 190 palabras). Viene en minúsculas y sin tildes.
- † 35.2% de las descripciones tiene más de 90 palabras, que es aproximadamente el límite de 128 *tokens* de los modelos de *embeddings* pequeños. Hay que verificarlo con el tokenizador del modelo elegido.

Información que no está en las columnas

Atributo mencionado	% de anuncios	¿Hay columna?
Vista	44.1	No
Balcón	30.1	No
Exterior †	30.1	No
Cercanía a colegio, TransMilenio o centro comercial	21.2	Parcial (distancias)
Remodelado	14.8	No
Piso N †	14.9	No (`piso` se excluyó: 100% N/A)
Iluminación	13.7	No
Dúplex o *penthouse* †	11.8	No
Área en metros †	49.4	Sí (redundante)
Terraza / ascensor	31.9 / 27.9	Sí (redundante)

Riesgos detectados en el texto †

- **Fuga del objetivo.** 1,190 descripciones (4.4%) contienen el precio publicado: la misma cifra, con ±2%, en pesos o en millones. Ejemplo: “venta $685 millones de pesos - negociables”. Además, 2.6% trae el valor de la administración y 0.9% el del canon de arriendo. Ambas cifras están muy ligadas al precio y no son características físicas. Un modelo de texto aprendería a leer esas cifras.
- **Señales del vendedor, no de la vivienda.** 10.5% usa términos de negociación (“negociable”, “oportunidad”, “remate”, “urge”). Ayudan a predecir el precio *publicado*, pero no dicen cuánto *debería* valer la vivienda dadas sus características.
- **Variables sensibles por otra vía.** 8.3% nombra su propio barrio, 8.9% su localidad y 6.4% el estrato. Si el modelo se estima sin estrato ni localidad para la sección de IA responsable, el texto puede reintroducirlos.
- **Duplicados.** 219 viviendas de prueba (4.1%) tienen una descripción idéntica a una de entrenamiento, y 1,249 filas comparten texto con otra. Es el mismo problema de los “gemelos”: un modelo de texto puede memorizar el anuncio.
- **Privacidad.** 2.0% incluye un celular o una mención de WhatsApp. Hay que enmascararlos antes de enviar textos a un servicio externo o de mostrarlos en diapositivas.

Una advertencia descriptiva. Dentro de cada estrato, la diferencia promedio en log(precio/m²) entre los anuncios que mencionan un atributo y los que no es pequeña y a veces tiene un signo inesperado: vista +0.038; remodelado −0.096, probablemente porque se remodelan edificios más viejos. Estas diferencias son correlaciones sin controles, no efectos. Lo que indican es que el aporte del texto, si existe, vendrá de combinaciones con las demás variables, y por eso hay que medirlo fuera de muestra, no suponerlo.

4. Alternativas

Cada alternativa se describe por cómo entra al modelo de precio. Todas comparten las reglas de la sección 6.

A. Texto con métodos clásicos (referencia, no aprendizaje profundo)

A1. Variables por palabras clave (diccionario). Columnas binarias del tipo "menciona vista / remodelado / exterior / piso alto / dúplex", y el número de piso extraído con una expresión regular. Se agregan al Random Forest actual.

- *Pros:* son explícitas, no se aprenden de los datos (no hay fuga), son interpretables, se pueden implementar en un día y amplían lo que ya hace `07_perfil_descripcion.py`.
- *Contras:* no capturan negaciones ("sin vista") ni sinónimos, y la lista de palabras la define el equipo. No cuenta como propuesta de aprendizaje profundo.

A2. TF-IDF (más reducción con SVD/PCA). Cada descripción se vuelve un vector TF-IDF. El vocabulario y los pesos IDF se ajustan solo en entrenamiento. Después se usa de una de dos formas: (i) en una regresión Ridge/Lasso sobre log(precio) junto a las variables tabulares, o (ii) se reduce a entre 20 y 50 componentes con PCA/SVD que entran al Random Forest.

- *Pros:* se vio en clase, es rápido y es el **punto de comparación para atribuir una mejora al aprendizaje profundo** y no al simple hecho de usar texto. La similitud coseno sobre TF-IDF también sirve para detectar descripciones casi idénticas (gemelos).
- *Contras:* ignora el orden y los sinónimos, y produce una matriz muy dispersa (≈9,000 palabras con 5 o más apariciones en entrenamiento †). Las componentes de SVD son difíciles de interpretar.

A3. Temas con LDA. Las proporciones de cada tema (por ejemplo, "lujo y amenidades", "conjunto familiar", "inversión") se usan como variables.

- *Pros:* es interpretable y se vio en clase.
- *Contras:* los temas no se eligen para predecir el precio. Suele aportar menos que TF-IDF y exige elegir el número de temas.

B. Redes neuronales entrenadas con nuestros datos

B1. *Embeddings* aprendidos y MLP híbrido. Una capa de *embedding* convierte cada palabra en un vector. Los vectores se promedian (bolsa de *embeddings*), se concatenan con las variables tabulares estandarizadas y pasan por un MLP con salida lineal para log(precio). Se entrena en PyTorch con minilotes y Adam/SGD.

- *Pros:* es exactamente lo que se vio (clases 7 a 9 y la Semana 6), se escribe a mano sin librerías nuevas además de PyTorch, y el texto y las variables tabulares se aprenden juntos.
- *Contras:* con 21,520 anuncios, los *embeddings* desde cero aprenden poco vocabulario. Hay que ajustar la arquitectura, la tasa de aprendizaje y la parada temprana, y una red sobre variables tabulares no suele superar a un Random Forest. **Por eso la comparación justa es MLP sin texto frente a MLP con texto, y ambos frente al Random Forest.**

B2. RNN / LSTM / GRU sobre la secuencia de palabras (CNN 1D como variante, Semana 7). El estado final de la red recurrente se concatena con las variables tabulares.

- *Pros:* usa el orden ("no tiene vista" frente a "tiene vista") y está en el temario (Semanas 8 y 10).
- *Contras:* el orden parece aportar poco para el precio: la mayoría de los anuncios son listas de atributos. Es más lenta y más difícil de ajustar que B1 y sufre el mismo problema de poco vocabulario. Las descripciones cortadas pierden el final.

C. Transformers preentrenados

C1. *Embeddings* preentrenados congelados. Un modelo de oraciones multilingüe o en español convierte cada descripción en un vector de 384 a 768 dimensiones. Ejemplos de la familia *sentence-transformers*: `paraphrase-multilingual-MiniLM-L12-v2` (384 dimensiones, 128 *tokens*) o `multilingual-e5-base` (768 dimensiones, 512 *tokens*). El modelo **no se entrena** con nuestros datos: se calcula una vez para las 26,900 viviendas y se guarda. Hay tres formas de usarlo:

- (a) PCA de los vectores (ajustado en entrenamiento), con entre 10 y 50 componentes que entran al Random Forest. Así se conserva el modelo actual y su lectura de importancia.
- (b) MLP híbrido (como B1, pero con el vector preentrenado en lugar de los *embeddings* aprendidos).
- (c) Modelo de residuos: una red o un Ridge que predice el error del Random Forest a partir del texto. Las predicciones del Random Forest deben ser fuera de fold dentro de entrenamiento.
- *Pros:* aprovecha conocimiento del idioma que 21,520 anuncios no pueden enseñar, entiende sinónimos y parte del contexto, y corre en el M1 de un integrante (inferencia de unos 27,000 textos cortos; hay que medir el tiempo, que debería ser de minutos). No hay fuga por el ajuste porque no usa nuestros precios. **Es lo que se propuso en la presentación de avance y corresponde al bloque de IA generativa (Transformers).**
- *Contras:* exige una librería no vista en clase. El vector no es interpretable, porque codifica todo, incluidas las señales del vendedor y los nombres de barrio. El texto en minúsculas y sin tildes difiere del texto con que se preentrenaron estos modelos, y con 128 *tokens* se truncaría cerca de un tercio de los anuncios.

C2. Ajuste fino (*fine-tuning*) de un Transformer en español (BETO, RoBERTa-BNE) con una cabeza de regresión y las variables tabulares concatenadas.

- *Pros:* es el más expresivo y adapta el modelo al vocabulario inmobiliario.
- *Contras:* tiene más de 100 millones de parámetros y 21,520 ejemplos (riesgo alto de sobreajuste). Además, el entrenamiento tarda horas por configuración, y la validación cruzada de 5 folds lo multiplica. Es difícil de reproducir en `00_main.py` dentro del tiempo que queda. No cabe antes del 25 de noviembre con la validación que exige `CLAUDE.md`.

D. IA generativa (LLM)

D1. LLM como extractor de características. A un LLM, por API (Semana 7), se le pasa cada descripción, **ya enmascarada y sin precio**, con un esquema fijo de salida en JSON:

- `vista` (exterior / interior / no dice)
- `piso` (número o nulo)
- `remodelado` y `para_remodelar`
- `duplex_penthouse`, `balcon` e `iluminacion`
- `area_privada_m2` si aparece
- `estado_conservacion`

Las columnas resultantes se agregan al Random Forest como cualquier otra variable.

- *Pros:*
  - **Es la alternativa que mejor conserva el objetivo paramétrico.** Convierte el texto en características que un comprador o vendedor podría llenar en un formulario, sin necesidad de un anuncio escrito.
  - Recupera `piso`, que se había excluido por 100% N/A.
  - Es interpretable: las nuevas columnas aparecen en la importancia de variables.
  - Se calcula una vez, sin usar el precio, así que no hay fuga por ajuste.
  - Cuesta poco. Con unos 700 *tokens* de entrada y 100 de salida por anuncio, son ≈19 M de entrada y ≈2.7 M de salida. A precios de lista de octubre de 2026, eso es **≈USD 3 con Claude Haiku 5.5** (≈USD 1.6 con la Batch API, que cuesta la mitad) o **≈USD 65 con Claude Sonnet 5.5** (≈USD 32 con Batch). El cálculo no incluye *tokens* de razonamiento, así que conviene usar esfuerzo bajo. Otros proveedores tienen precios del mismo orden.

- *Contras:*
  - El LLM puede inventar o equivocarse. **Hay que validarlo** contra una muestra de unas 200 descripciones etiquetadas a mano por el equipo, reportando precisión y *recall* por campo.
  - Exige fijar el modelo, el *prompt* y la temperatura, y guardar las salidas en `01_Datos/02_Limpios/` para que la corrida sea reproducible sin volver a llamar a la API.
  - Envía texto a un servicio externo: los anuncios son públicos, pero se enmascaran los teléfonos.
  - Por sí solo no es una red entrenada por el equipo. Para la rúbrica se presenta como complemento de IA generativa de C1, no como sustituto.

D2. LLM que predice el precio directamente (*zero-shot* o *few-shot*: "¿cuánto vale este apartamento?").

- *Pros:* no requiere entrenamiento y sirve como curiosidad comparativa.
- *Contras:* no se puede validar con CV ni reproducir. El LLM pudo haber visto estos anuncios (*scraping* público de 2024) y recordar el precio: la contaminación invalida la prueba. No es interpretable y cuesta más por anuncio. **Se descarta.**

E. Imágenes (fuera de alcance)

El diccionario de la fuente menciona un `images.csv` con URLs de fotos, pero está comentado y no hace parte de los crudos que tenemos. Un modelo multimodal (CNN sobre las fotos) queda fuera de alcance; se puede mencionar como trabajo futuro.

5. Comparación frente al objetivo

Criterios:

- **DL/IAG:** si cumple el criterio de la rúbrica (llevar el problema a aprendizaje profundo o IA generativa).
- **Curso:** si se vio en clase.
- **Info nueva:** capacidad de aportar información sobre el precio que no esté en las columnas.
- **Paramétrico:** si el modelo final se puede usar describiendo la vivienda con características, sin un texto de anuncio.
- **Interpr.:** interpretabilidad.
- **Riesgo:** exposición a fuga y sesgo por las señales del vendedor, el barrio o el estrato presentes en el texto.
- **Esfuerzo:** días-persona aproximados, con la evaluación completa.

Alternativa	DL/IAG	Curso	Info nueva	Paramétrico	Interpr.	Riesgo	Costo	Esfuerzo
A1 Palabras clave	No	Sí	Baja–media	Sí	Alta	Bajo	Nulo	1
A2 TF-IDF (+SVD)	No	Sí	Media	No	Media	Medio	Nulo	1–2
A3 LDA	No	Sí	Baja	No	Alta	Medio	Nulo	1–2
B1 *Embeddings* propios + MLP	Sí	Sí	Media	No	Baja	Medio	Bajo	3–4
B2 LSTM/GRU	Sí	Sí	Media	No	Baja	Medio	Medio	4–5
C1 Transformer congelado	Sí	Parcial	Media–alta	No	Baja	Alto	Bajo	3–4
C2 *Fine-tuning* Transformer	Sí	Parcial	Alta	No	Baja	Alto	Alto	7+
D1 LLM extrae características	Sí (IAG)	Parcial (APIs)	Media	**Sí**	**Alta**	Bajo*	≈USD 2–65	3–4

Alternativa	DL/IAG	Curso	Info nueva	Paramétrico	Interpr.	Riesgo	Costo	Esfuerzo
D2 LLM predice el precio	Sí (IAG)	Parcial	Incierta	No	Nula	Muy alto	Medio	1–2

* Bajo siempre que el esquema de extracción no incluya barrio, estrato ni cifras de dinero.

Lectura de la tabla frente al objetivo principal

- **El texto es un insumo con dos caras.** Trae calidad de la vivienda (vista, piso, remodelación) que las columnas no tienen, pero también trae el precio mismo, la estrategia del vendedor y la ubicación. Las alternativas que usan el texto completo (A2, B, C) aprenden de todo eso a la vez. Las que lo convierten en características definidas (A1, D1) solo toman lo que corresponde a la vivienda.
- **En el uso real, un comprador o vendedor describe una vivienda, no siempre un anuncio.** A1 y D1 producen un modelo que se puede alimentar con un formulario. C1 y B exigen un texto libre al predecir, y la predicción cambia según cómo esté redactado.
- **La rúbrica pide aprendizaje profundo o IA generativa justificado por la estructura de los datos.** C1 lo cumple de forma directa y D1 como IA generativa. A1 y A2 son necesarios como referencia, pero por sí solos no cumplen.
- **La interpretabilidad fue una razón para elegir el Random Forest.** C1(a) y D1 la conservan porque entran al mismo bosque. Las variantes con MLP la pierden.

6. Reglas de diseño comunes (cualquier alternativa)

1. **Misma partición y misma prueba.** Se usa `particion_train_test.csv`, la CV de 5 folds con el `KFold` fijo (`random_state=2026`) y la prueba solo al final. Todo modelo se reporta junto a la línea base y al Random Forest tabular: RMSE y MAPE en entrenamiento, CV y prueba, en pesos.
2. **Limpieza del texto antes de cualquier modelo.**
   - Enmascarar con expresiones regulares las cifras de dinero (precio, administración, canon), por ejemplo reemplazándolas por un marcador `<cifra_dinero>`, y los teléfonos.
   - Reportar cuántas descripciones se modificaron, como en toda limpieza.
   - Decidir con el equipo si además se enmascaran los nombres de barrio.
3. **Transformaciones aprendidas del texto solo con entrenamiento.** El vocabulario y el IDF de TF-IDF, el PCA/SVD, el LDA, el escalador y los *embeddings* propios se ajustan con entrenamiento y la prueba solo se transforma. Como no usan el precio, `CLAUDE.md` permite ajustarlos con todo el entrenamiento para la CV. Las partes supervisadas (la red, el modelo de residuos) se entrenan dentro de cada fold.
4. **Lo que no se ajusta con nuestros datos se calcula una vez.** Los *embeddings* preentrenados (C1) y la extracción con LLM (D1) se calculan para las 26,900 viviendas y se guardan en `01_Datos/02_Limpios/`, con el nombre del modelo y su versión. No hay fuga mientras la entrada no incluya el precio.
5. **Ablaciones para atribuir el efecto al texto.**
   - Random Forest tabular frente a Random Forest más texto.
   - MLP sin texto frente a MLP con texto.
   - Las dos versiones del modelo con texto, con y sin estrato y localidad, para ver si el texto reintroduce la ubicación (sección de IA responsable).
6. **Sensibilidad.** Métricas de prueba sin los gemelos (como en `05_sensibilidad_gemelos.csv`) y sin las 219 descripciones duplicadas entre entrenamiento y prueba.
7. **Desagregación por estrato y localidad.** Mismas tablas que `05_errores_por_*`.
8. **Redes neuronales.**
   - Semilla fija (`torch.manual_seed(2026)`).
   - Parada temprana con un subconjunto de validación sacado de entrenamiento, no de la prueba.

- Objetivo en log y en niveles, según la decisión del equipo. En niveles hay que estandarizar el objetivo para que el entrenamiento sea estable.

9. **Reproducibilidad.**
   - Un script numerado por tarea, por ejemplo `09_texto_limpieza.py`, `10_texto_embeddings.py` y `11_modelos_texto.py`.
   - Versiones fijas en `requirements.txt`.
   - Los insumos costosos (*embeddings*, extracción con LLM) se guardan en disco para que `00_main.py` no los recalcule ni llame a la API en cada corrida.
10. **Uso de IA documentado.** Si se usa un LLM, en la bitácora queda qué modelo y qué *prompt* se usaron, para qué y cómo se validó.

No se anticipa cuánto bajará el error. Un resultado en que el texto no reduzca el RMSE de CV también es informativo y se reporta igual.

7. Recomendación y plan hasta el 25 de noviembre

Paso	Qué	Archivo que produce	Terminado cuando
1	Limpieza y enmascaramiento del texto, más variables por palabras clave (A1)	`09_texto_limpieza.py` → `descripcion_limpia.csv`	Tabla con el número de textos modificados; RF + A1 con CV y prueba
2	Referencia TF-IDF (A2)	Dentro de `11_modelos_texto.py`	RMSE/MAPE de CV y prueba frente al RF tabular
3	*Embeddings* preentrenados (C1), más PCA → RF y MLP híbrido en PyTorch	`10_texto_embeddings.py` (vectores en disco) y `11_modelos_texto.py`	Tabla con MLP sin texto, MLP con texto, RF y RF + C1, por estrato
4	Extracción con LLM (D1), validada contra 200 etiquetas manuales	`12_texto_llm.py` → `atributos_llm.csv`	Precisión/*recall* por campo; RF + D1 con CV y prueba
5	Comparación final y sesgos	Tablas 11_∗, 12_∗ y figura para la presentación	Todas las variantes contra la línea base en una tabla

Reparto sugerido (el criterio de trabajo en equipo evalúa commits individuales): un integrante para los pasos 1–2, otro para el paso 3 y otro para el paso 4. El paso 5 se hace en conjunto.

Si el tiempo no alcanza, el orden de prioridad es 1 → 3 → 4 → 2. El paso 1 es requisito para todo lo demás. El paso 3 es el que cumple la propuesta presentada. El paso 2 se puede reemplazar por A1 como referencia clásica.

Preguntas abiertas para el equipo

- Aprobar las librerías nuevas: `torch`, `sentence-transformers` o `transformers` y el SDK del proveedor de LLM (regla de `CLAUDE.md`).
- Para las descripciones con precio literal: ¿enmascarar la cifra (recomendado) o eliminar la fila? ¿Se enmascaran también los nombres de barrio?
- ¿Presupuesto y proveedor para la API de D1? ¿Quién etiqueta la muestra de validación y con qué criterio?
- ¿Las redes se estiman también en niveles o solo en log? La decisión vigente es estimar ambas escalas.
- ¿Qué modelo de *embeddings*? Hay una disyuntiva entre la longitud máxima (128 frente a 512 *tokens*) y el tiempo de cómputo.

8. Uso de IA en esta nota

- **Herramienta:** Claude Code (modelo Claude Opus 5.5), a pedido de Santiago Muñoz.
- **Para qué:**
  - Revisar el programa del curso, las diapositivas disponibles, el guion de la presentación de avance y las tablas de resultados.
  - Correr una exploración preliminar del texto.
  - Redactar la comparación.
- **Cómo se verificó:**
  - Las cifras de los modelos salen de `04_metricas_modelos.csv` y las del perfil de texto de `07_perfil_descripcion.csv`.
  - Las cifras marcadas con † salen de un script exploratorio con expresiones regulares, no versionado; son aproximadas y deben reproducirse con un script numerado antes de usarlas en la presentación.
  - Los precios de la API son de lista a octubre de 2026 y deben confirmarse al momento de usarla.
  - Los nombres de modelos de *embeddings* y sus límites de *tokens* deben confirmarse en su documentación antes de elegir.